

The Two Pointers Approach

A fundamental technique for efficient array and string manipulation in Data Structures & Algorithms.

| What is the Two Pointers Technique?

Core Concept

The technique involves using two distinct pointers (indices) to traverse a data structure—typically an array or string—simultaneously.

It transforms naive nested loop solutions into efficient linear scans.

Efficiency & Goal

Goal: Efficiently search for pairs, reverse data, or track a window.

Complexity: Reduces time complexity from $O(n^2)$ to $O(n)$.

Space: usually $O(1)$ (in-place).

| Two Main Implementation Strategies

1. Opposite Directions

Mechanism: Pointers start at opposite ends (`left = 0` , `right = n-1`) and move towards each other.

Use Cases:

- Checking Palindromes
- Two Sum in Sorted Arrays
- Reversing an Array

2. Same Direction

Mechanism: Both pointers start at the beginning (`slow` , `fast`) and move in the same direction at different rates.

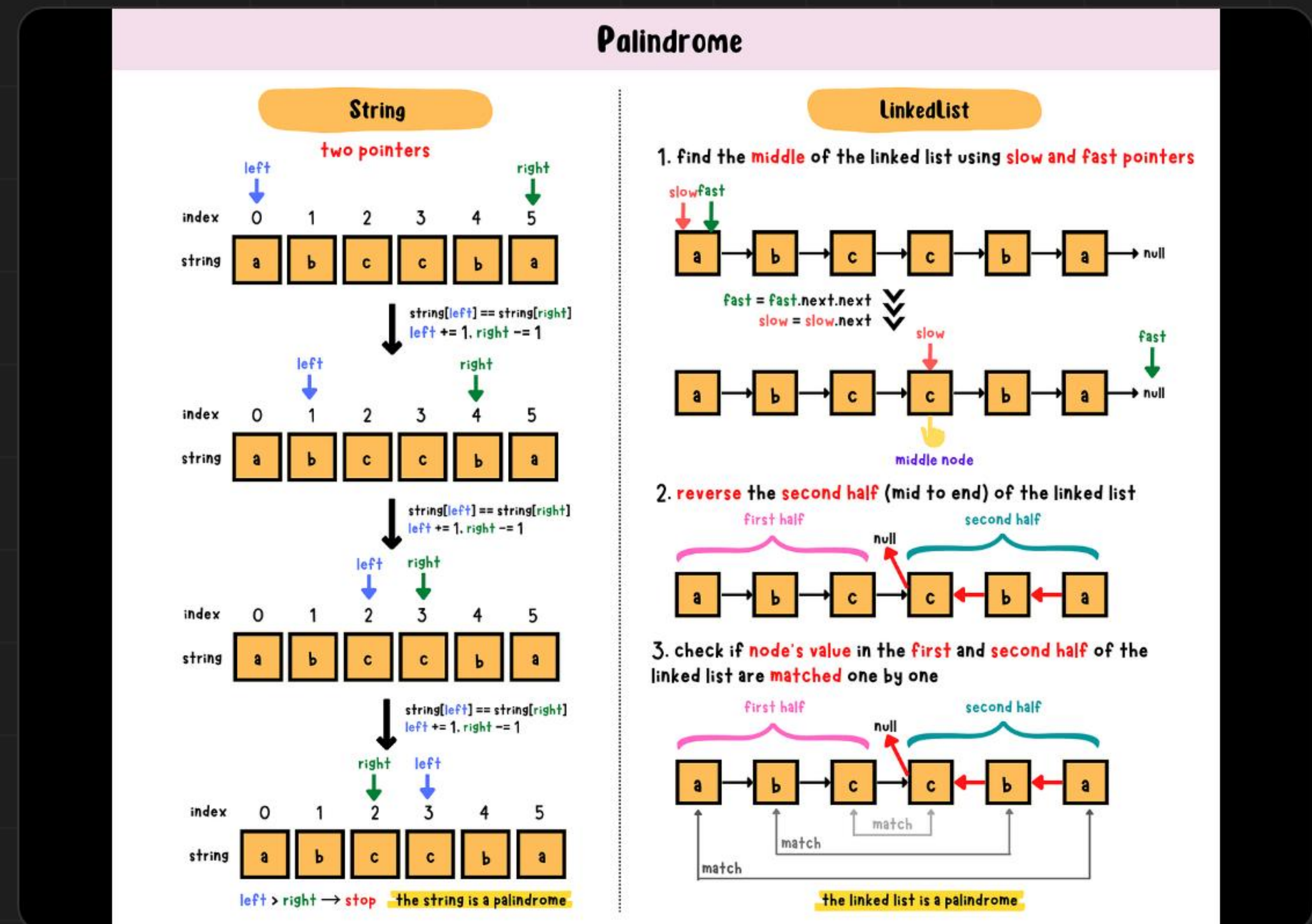
Use Cases:

- Removing Duplicates
- Sliding Window Problems
- Cycle Detection (Linked List)

Example 1: Palindrome Checker

Strategy: Meet in Middle

- Initialize `left` at index 0 and `right` at index $n-1$.
- Loop while `left < right`.
- Compare characters at both pointers.
- If mismatch: return `False`.
- Else: `left++`, `right--`.
- If loop finishes: return `True`.



Example 2: Two Sum (Sorted Array)

Strategy: Guided Search

Find two numbers that add up to a `target`.

- Calculate `sum = arr[left] + arr[right]`.
- If `sum == target`: **Found**.
- If `sum < target`: We need a larger sum, so move `left` to the right (increment).
- If `sum > target`: We need a smaller sum, so move `right` to the left (decrement).

01
Step

To apply two pointer technique, we make sure that array is sorted.

Unsorted array =

1	4	45	6	10	-8
0	1	2	3	4	5

Sorted array =

-8	1	4	6	10	45
0	1	2	3	4	5

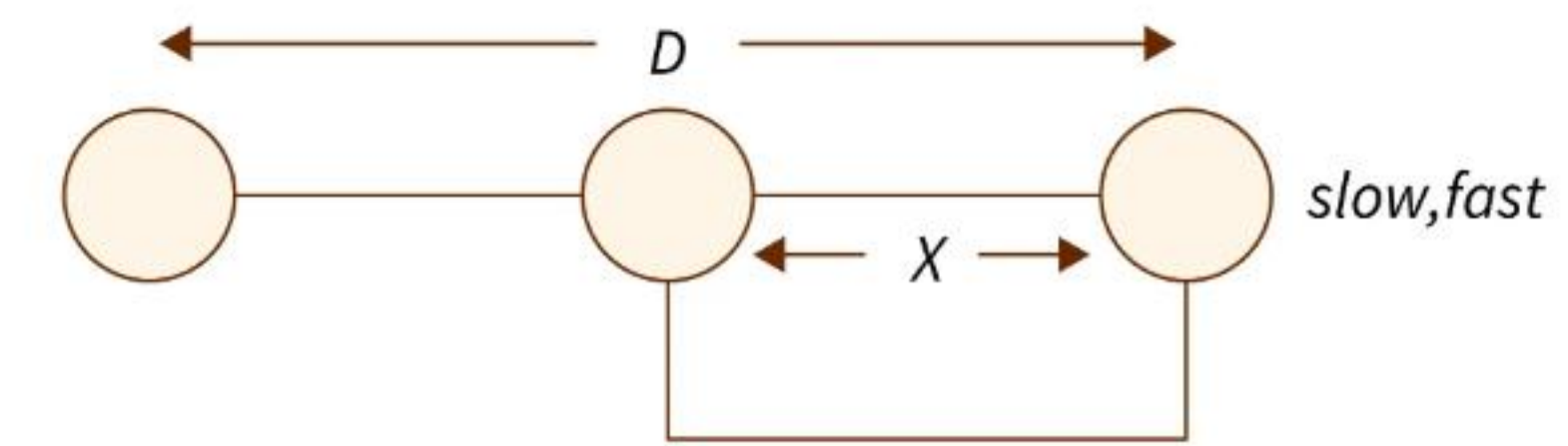
Two Pointer Technique

Example 3: Remove Duplicates

Strategy: Fast & Slow Pointers

Modify the array in-place to keep unique elements.

- `slow` tracks the last unique element.
- `fast` scans ahead for new values.
- If `arr[fast]  $\neq$  arr[slow]`:
 - Increment `slow`.
 - Update `arr[slow] = arr[fast]`.



Cycle length = c

SCALER
Topics

Advanced: Sliding Window

Definition

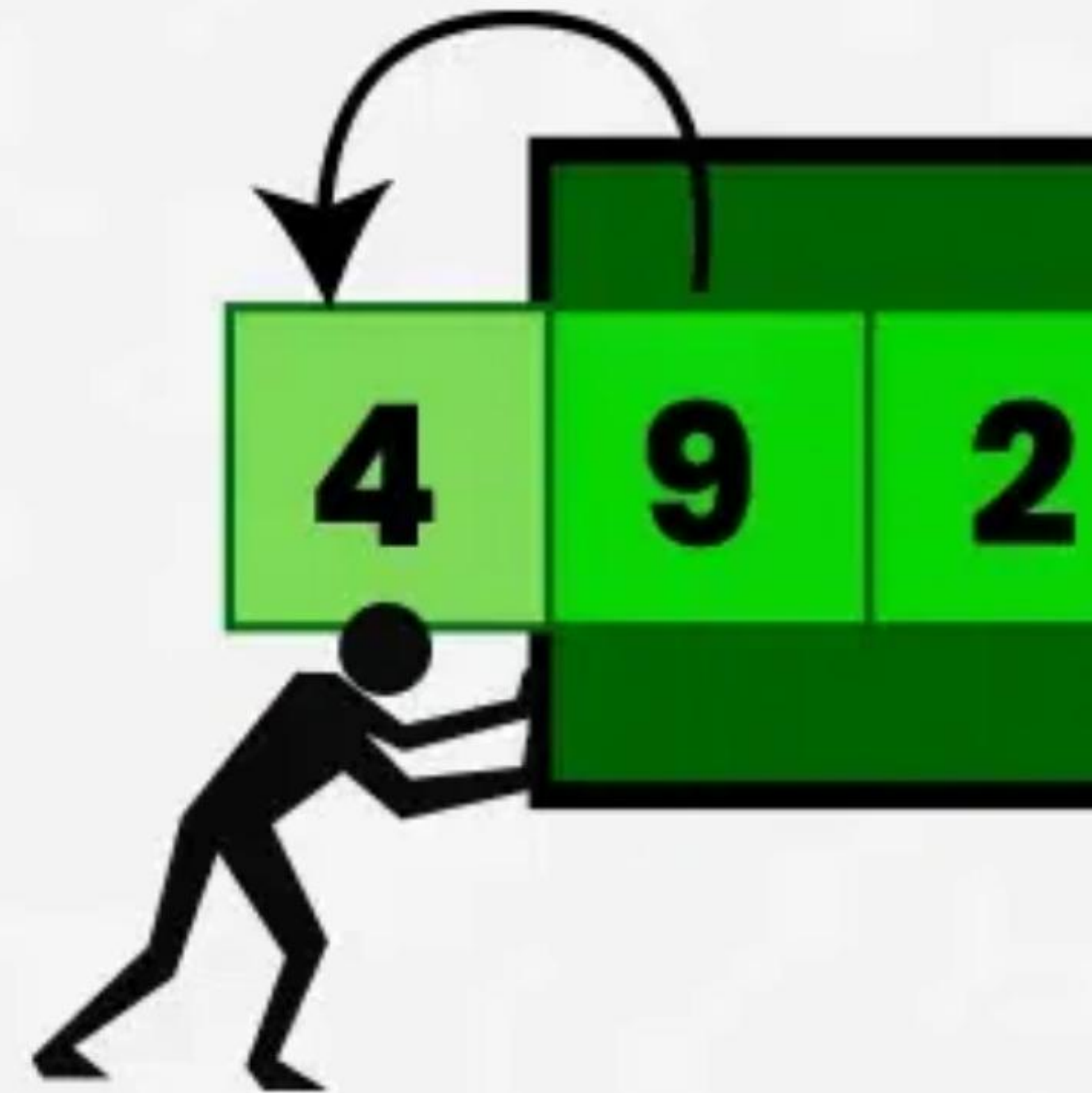
A variation where two pointers define a "window" (sub-array) that grows and shrinks to satisfy specific conditions.

Mechanism

1. **Expand:** Move **end** pointer to grow window.
2. **Shrink:** Move **start** pointer to optimize or validate.

Example: "Find the max sum subarray of size K".

NG
OW
QUE



Complexity Analysis

Feature	Two Pointers Approach	Naive Approach (Nested Loops)
Time Complexity	$O(n)$ (Linear)	$O(n^2)$ (Quadratic)
Space Complexity	$O(1)$ (Constant)	$O(1)$ or $O(n)$
Best For	Sorted arrays, Linear structures	Unsorted data (brute force)
Data Access	Sequential, Single Pass	Repeated scanning

Questions?

Thank you for listening.

Practice Problems:

Trapping Rain Water • Container With Most Water • 3Sum

Mastering the Sliding Window

An essential technique for optimizing subarray and substring problems in DSA.

What is Sliding Window?

The "Window" Metaphor

Imagine a rectangular frame sliding over a sequence of data (like an array or string). At any given time, you can only see the elements inside the frame.

This frame can **slide** (move forward), **grow**, or **shrink**.

The Optimization Power

Brute Force: Nested loops often lead to $O(n^2)$ complexity by re-calculating overlapping data.

Sliding Window: Converts this to $O(n)$ by reusing the result of the previous window (e.g., subtracting the element that left and adding the one that entered).

How it Moves



Expand

The **Right Pointer** moves forward to include new elements into the window. This typically happens to find a valid condition.



Shrink

The **Left Pointer** moves forward to exclude elements. This happens to optimize the window (e.g., make it smaller) or restore validity.



Validate

At each step, we check if the current window satisfies the problem constraints (sum, unique chars, etc.) and update the result.

Fixed vs. Variable Size

Fixed Size Window

The window size `k` is constant.

Movement: Slide the window one step at a time.

Pattern: Add `arr[i]`, Remove `arr[i-k]`.

Example: Max sum of 3 consecutive elements.

Variable Size Window

The window grows and shrinks dynamically.

Movement: Expand `right` until condition is met (or broken), then shrink `left` to optimize.

Pattern: `while (condition) { remove
arr[left]; left++; }`

Example: Longest substring with no repeats.

Problem 1: Max Sum Subarray (Fixed)

Goal: Find max sum of size K

Given `arr = [2, 1, 5, 1, 3, 2]` and `k = 3`.

- **Initial Window:** `[2, 1, 5]` Sum = 8.
- **Slide 1:** Add 1, Remove 2. New Sum = $8 + 1 - 2 = 7$.
- **Slide 2:** Add 3, Remove 1. New Sum = $7 + 3 - 1 = 9$.
- **(Max)**
- **Benefit:** Avoids re-summing the middle elements every time.

Example

Consider the array:

11	-8	16	-7	24	-2	3
----	----	----	----	----	----	---

k = 3

Subarrays of size k = 3:

11	-8	16	-8	16	-7	16	-7	24	-7	24	-2	24	-2	3
Sum = 19			Sum = 1			Sum = 33			Sum = 15			Sum = 25		

Maximum Average Subarray of size 3

Deserve ©

Problem 2: Longest Unique Substring

Goal: Longest substring w/o repeats

String: `"abcabcbb"`

- **Expand:** `right` moves. Window: `"abc"`.
- **Conflict:** Next is 'a'. Duplicate found!
- **Shrink:** Move `left` past the first 'a'. Window becomes `"bca"`.
- **Result:** Track the maximum window size seen (3).

 Visualization of sliding window for longest substring without repeating characters

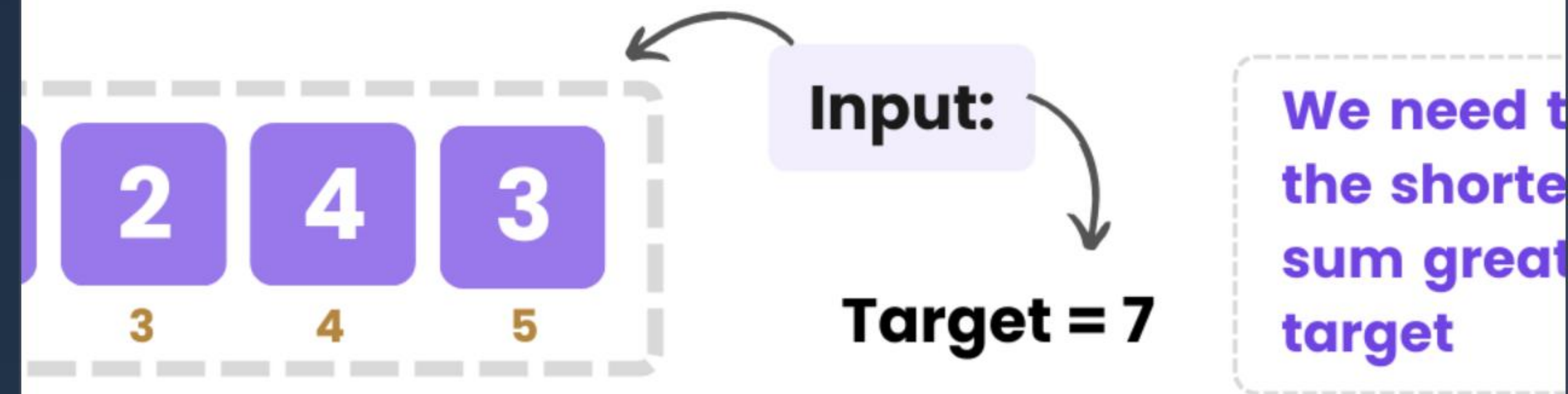
Problem 3: Min Size Subarray Sum

Goal: Smallest subarray with $\text{Sum} \geq \text{Target}$

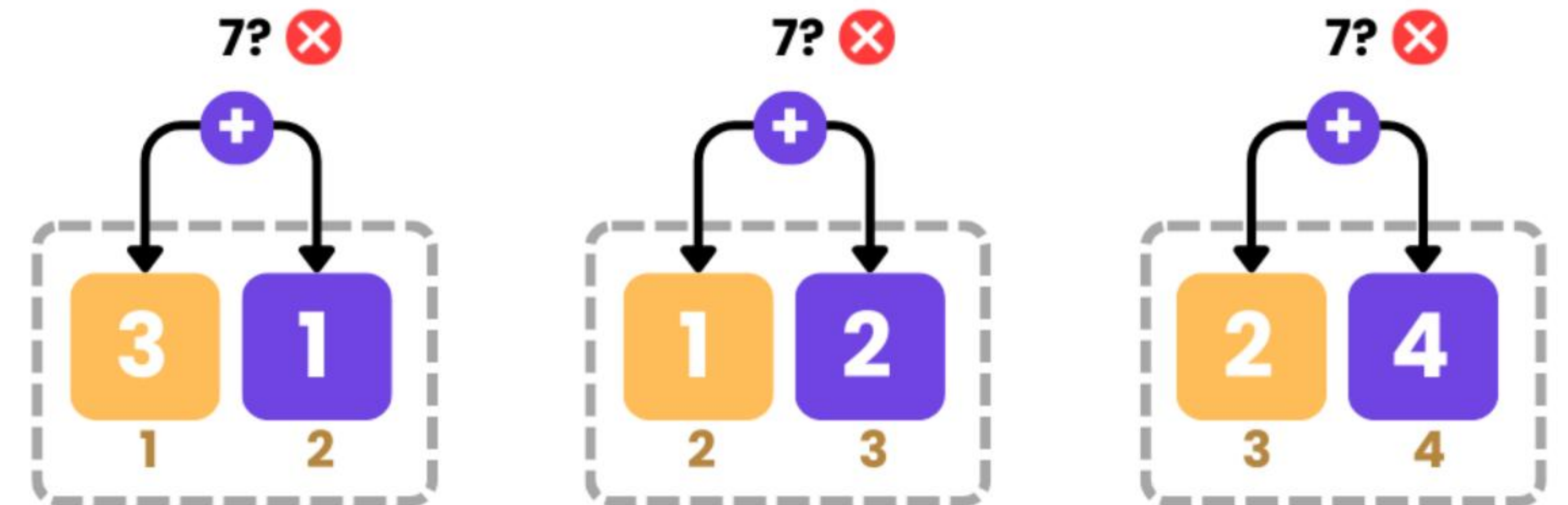
Strategy (Variable Window):

1. Expand: Add elements to `sum` until `sum >= target`.
2. Shrink: While `sum >= target`, record length, subtract `arr[left]`, and increment `left` to see if a smaller window works.

This finds the minimal length efficiently.



If the target is not already present in the array, We will check subArray



shortest subarray that meets the condition has length = 2. Thus, we did not proceed with length = 3.

Output =

Complexity Analysis

FEATURE	BRUTE FORCE	SLIDING WINDOW
Time Complexity	$O_n \times k$ or O_n^2	O_n (Linear)
Space Complexity	O_1	O_1 or O_k (for sets/maps)
Operations	Redundant recalculations	Incremental updates
Pointers Used	Nested loops (i, j)	Two pointers (left, right)

Questions?

Thank you!

Next Topic: Two Pointers vs Sliding Window Differences

// DSA LECTURE SERIES

ADVANCED ALGORITHMIC CHALLENGES

Mastering Hard Two Pointers & Sliding Window
Problems.

PROBLEM 1: MINIMUM WINDOW SUBSTRING

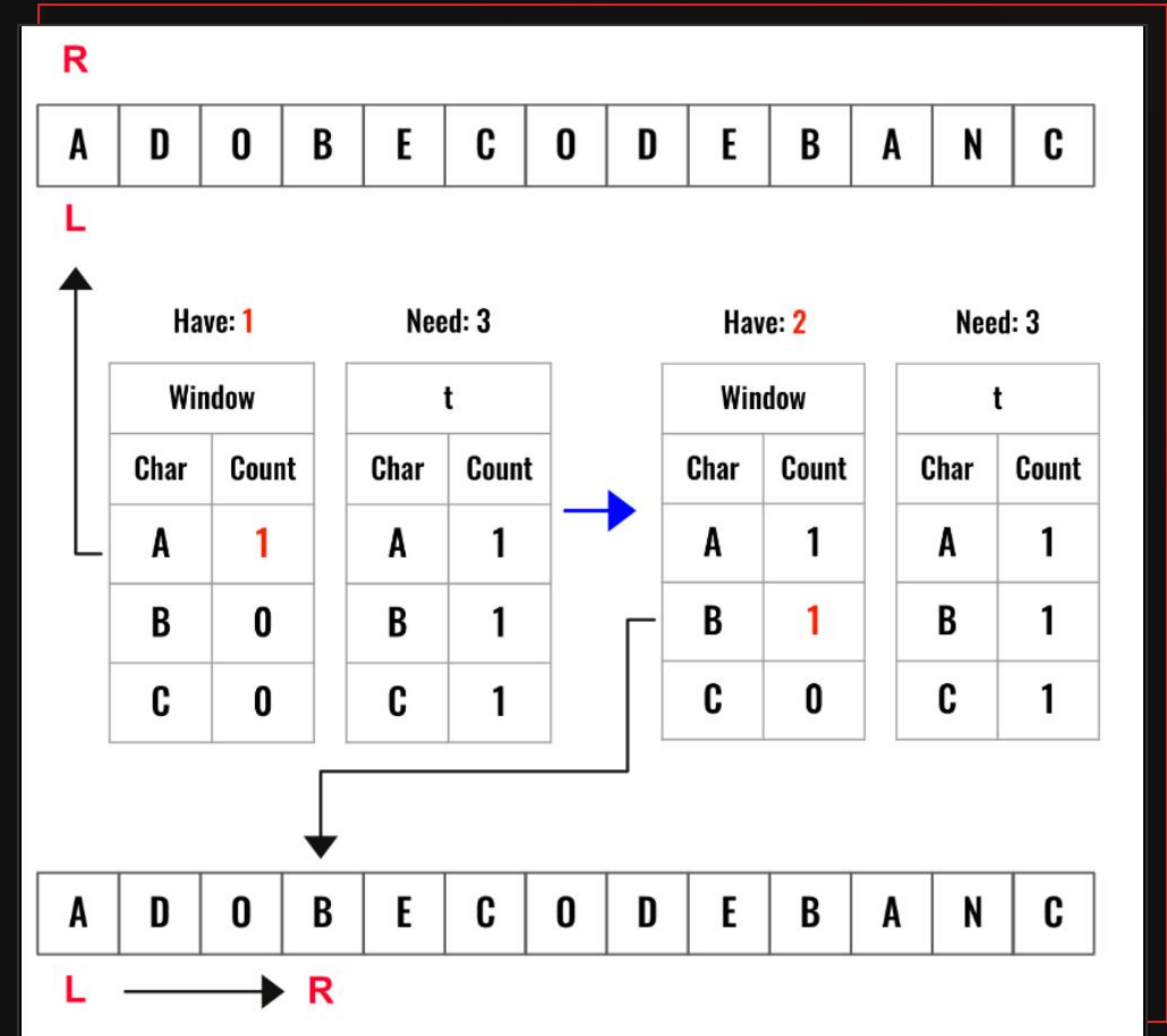
PROBLEM

Given strings `s` and `t`, find the minimum window in `s` which will contain all the characters in `t`. If no such window exists, return "".

Difficulty: **HARD**

SOLUTION STRATEGY

- > Use two pointers `left`, `right` and a hash map for character counts of `t`.
- > **Expand:** Move `right` to include characters until the window is valid (has all chars).
- > **Shrink:** While valid, move `left` to minimize length and update the global minimum.
- > **Complexity:** Time $O(N)$ | Space $O(1)$ (char set is fixed).



PROBLEM 2: TRAPPING RAIN WATER

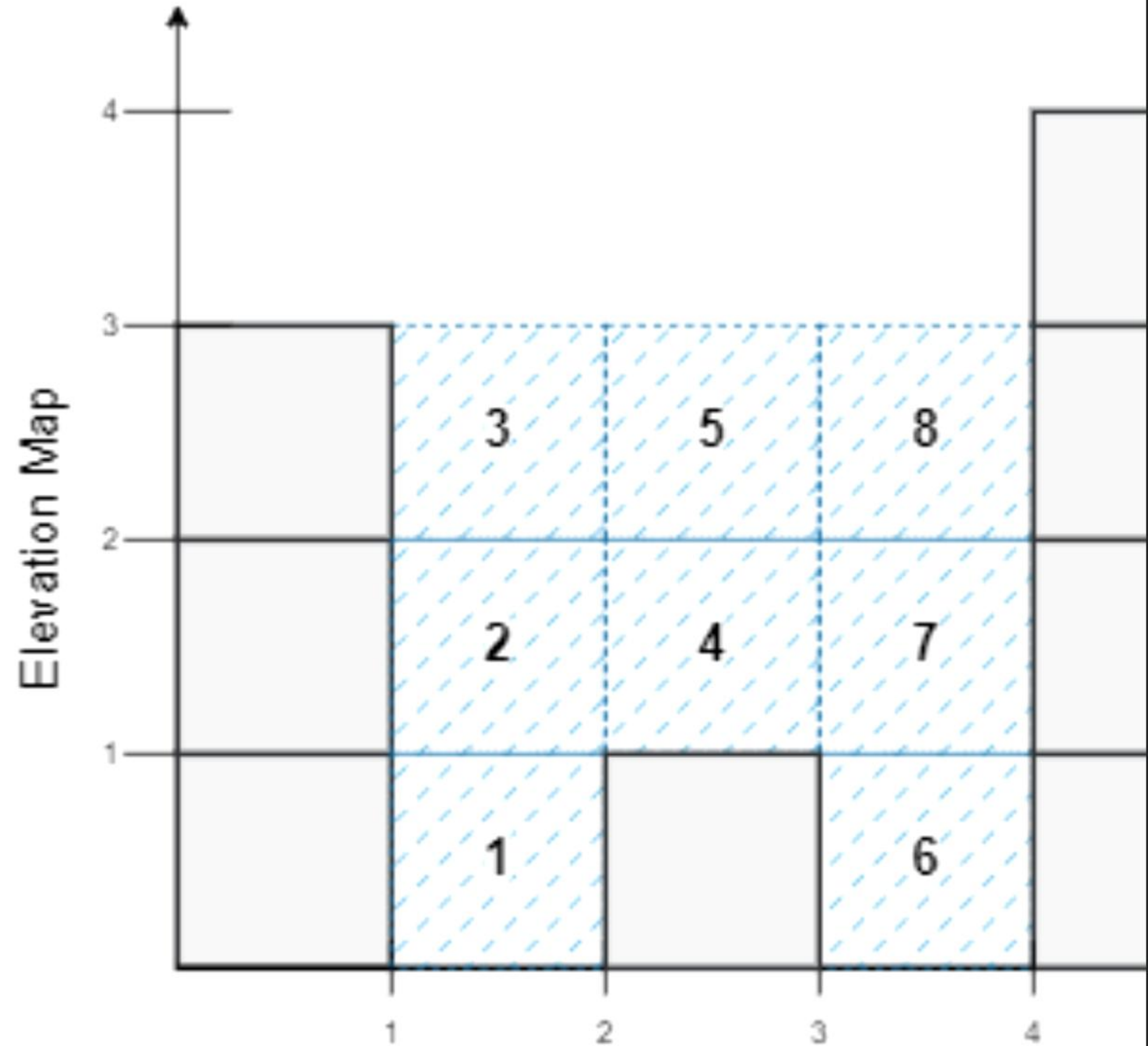
PROBLEM

Given `n` non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Difficulty: **HARD**

TWO POINTERS SOLUTION

- > Maintain `left` and `right` pointers at ends, and track `maxLeft` and `maxRight`.
- > Compare heights: if `height[left] < height[right]`, process left side.



ANY QUESTIONS?

// END OF SESSION

Review these patterns. They are frequent in top-tier tech interviews.